Computer Vision Homework 6

Name: NICKHOLAS

Student ID: M11515801

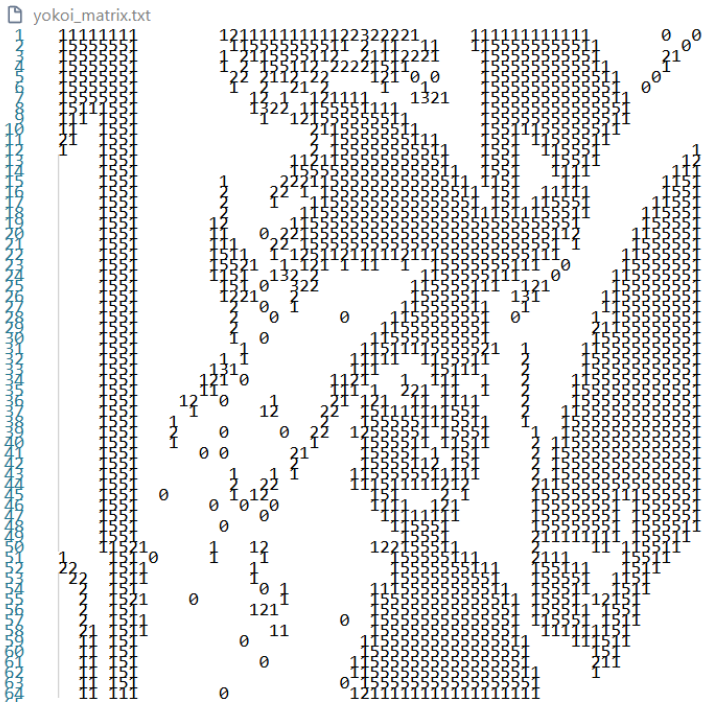
Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement the Yokoi connectivity number requirement.

Using OpenCV (cv2) to read the image, and the processing is done on NumPy arrays. The Yokoi connectivity number is computed on a binarized lena.bmp that is downsampled from 512x512 to 64x64, using 4-connectivity.

Below are the results and explanations:



Explanation:

The preprocessing processes, binarization at the threshold 128 and downsampling image from 512x512 to 64x64 were conducted. The binarized image was divided into 8x8 blocks, and the topmost-left pixel of each block was taken as the downsampled data.

```
# Binarize lena
def binarize(img):
    h, w = img.shape
    binary = np.zeros((h, w), dtype=np.uint8)
    for i in range(h):
        for j in range(w):
            binary[i][j] = 255 if img[i][j] >= 128 else 0
    return binary
```

```
def downsample(img, block_size=8):
    h, w = img.shape
    new_h = h // block_size
    new_w = w // block_size
    down = np.zeros((new_h, new_w), dtype=np.uint8)
    for i in range(new_h):
        for j in range(new_w):
            down[i][j] = img[i * block_size][j * block_size]
    return down
```

Before jumping to the Yokoi connectivity numbers function, below are the fragment of the codes for h function and f function.

```
def h_func(b, c, d, e):
    if b == c:
        if d != b or e != b:
            return 'q'
        else:
            return 'r'
    else:
        return 's'

def f_func(a1, a2, a3, a4):
    if a1 == 'r' and a2 == 'r' and a3 == 'r' and a4 == 'r':
        return 5
    else:
        return [a1, a2, a3, a4].count('q')
```

Then, the Yokoi connectivity numbers were counted by using 4-connected, and the pixels outside the image were treated as background. The implementation details of the counting algorithm follow the course's lecture slides.

```python
def yokoi_connectivity(img):
    h, w = img.shape
    result = np.full((h, w), -1, dtype=int)

    for i in range(h):
        for j in range(w):
            if img[i][j] == 0:
                continue

            # x0 = center (i,j), neighbors in 4-connected + diagonals
            # x0 = (i,j), x1 = (i,j+1), x2 = (i-1,j), x3 = (i,j-1), x4 = (i+1,j)
            # x5 = (i+1,j+1), x6 = (i-1,j+1), x7 = (i-1,j-1), x8 = (i+1,j-1)
            x0 = img[i][j]
            x1 = get_pixel(img, i, j + 1)      # right
            x2 = get_pixel(img, i - 1, j)      # top
            x3 = get_pixel(img, i, j - 1)      # left
            x4 = get_pixel(img, i + 1, j)      # bottom
            x5 = get_pixel(img, i + 1, j + 1)  # bottom-right
            x6 = get_pixel(img, i - 1, j + 1)  # top-right
            x7 = get_pixel(img, i - 1, j - 1)  # top-left
            x8 = get_pixel(img, i + 1, j - 1)  # bottom-left

            # h for four quadrants
            a1 = h_func(x0, x1, x6, x2)  # quadrant 1: right, top-right, top
            a2 = h_func(x0, x2, x7, x3)  # quadrant 2: top, top-left, left
            a3 = h_func(x0, x3, x8, x4)  # quadrant 3: left, bottom-left, bottom
            a4 = h_func(x0, x4, x5, x1)  # quadrant 4: bottom, bottom-right, right

            result[i][j] = f_func(a1, a2, a3, a4)

    return result
```

The result is a 64x64 matrix as shown above, in which the background is left blank.